SD – LAB nr 1 - 25.02.2026 - Materiały

📋 Agenda zajęć

Część A: Sprawy Organizacyjne

- Wprowadzenie od prowadzącego i runda studencka (imię, oczekiwania)
- Omówienie harmonogramu, kontraktu i zasad zaliczenia.

Część B: Warsztat „Pierwsza Krew – Węzeł Akwizycji”

- **Intro:** Wprowadzenie do motywu przewodniego zajęć - projekt IoT i jak nad nim pracujemy. Omówienie płytki
- **Zadanie BASIC:** Setup środowiska, "Hello World" (Blink) oraz "Hello Observability" (Serial Monitor).
- **Zadanie PERFORMANCE:** Podłączenie czujnika temperatury DS18B20 i budowa pętli typu *fail-safe*.
- **Check-off & Audit Trail:** Weryfikacja postępów, Push kodu przed wyjściem. Czyszczenie poświadczeń Git. Co na następne laby.

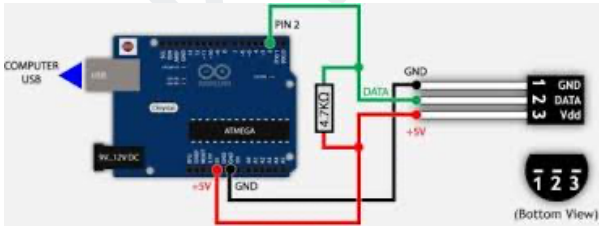
Artefakt: Działająca w przeglądarce aplikacja "Hello UniDesk" wrzucona na repozytorium GitHub.

UKW/WI – PPC

🛠️ Zadania - Projekt: Węzeł Telemetryczny Edge

Motyw przewodni: Zanim wyślemy dane w świat, musimy mieć pewność, że nasz węzeł brzegowy (ESP8266) jest stabilny, a środowisko deweloperskie działa bez zarzutu. Zrozumienie, co dzieje się na mikrokontrolerze (Observability), to fundament.

Praca na zajęciach: Działasz samodzielnie jako **Inżynier**, ale w razie blokady stosujemy **Team Spirit** – pomagacie sobie nawzajem, zanim ja dotrę do Waszego stanowiska. Błąd to informacja, nie porażka.



🧠 Zanim zaczniesz: Twój Hardware na dziś (Wemos D1 / ESP8266)

Zanim napiszemy pierwszą linijkę kodu, zróbmy szybki audyt tego, co leży przed Tobą na biurku. Musisz wiedzieć, z czym pracujesz, żeby nie traktować sprzętu jak czarnej skrzynki.

1. Płytka D1 (Mózg operacji: ESP8266) To Twój mikrokontroler – węzeł brzegowy (Edge Node) naszego systemu. Jest mały, ale posiada 32-bitowy procesor i wbudowany moduł Wi-Fi. Jednak uwaga: **dzisiaj pracujemy wyłącznie "po kablu" (Wired)**. Skupiamy się na logice pętli i odczytach. Moduł radiowy uruchomimy dopiero na kolejnych zajęciach (Lab 2), by wypuścić dane w świat.

2. Czym w ogóle jest "Arduino"? Zapewne słyszałeś to hasło. W naszym inżynierskim żargonie na tych zajęciach "Arduino" **NIE** oznacza niebieskiej płytki ze szkoły. Dla nas to **framework programistyczny** – potężny zbiór gotowych funkcji w C++. Dzięki niemu nie musisz pisać w czystym asemblerze, by zapalić diodę lub odczytać pin. Piszemy kod w C++ z wykorzystaniem frameworka Arduino, który PlatformIO przetłumaczy na zrozumiały język dla procesora ESP8266.

3. Twój dzisiejszy arsenał roboczy:

- **Kabel USB:** Dziś to Twoja "pępowina". Robi dwie rzeczy naraz: dostarcza zasilanie do płytki i pozwala na dwukierunkową komunikację tekstową (Serial Monitor) z Twoim komputerem.
- **Czujnik DS18B20:** Cyfrowy sensor temperatury. To on dostarczy nam realne dane do analizy.
- **Kable stykowe (Dupont):** Przewody, którymi fizycznie zamkniesz obwód między zasilaniem, procesorem a czujnikiem. Zero lutowania, czysty plug & play.

Masz sprzęt? Rozumiesz plan? To przechodzimy do kodu.

UKW/WI – PRC

🟢 BASIC: (MUST - Wymagane)

Cel: Skonfigurowanie repozytorium, inicjalizacja projektu PlatformIO, wgranie kodu migającego wbudowaną diodą oraz poprawne logowanie informacji przez port szeregowy.

Akcja 1 (Higiena Git): Ustaw swoją tożsamość Git (Imię, Nazwisko, Email) lokalnie w folderze roboczym (użyj `git config --local`). Pamiętaj aby nie zostawić swoich danych na publicznym PC.
Akcja 2 (Generowanie kodu & IDE): Otwórz VS Code i użyj rozszerzenia **PlatformIO**. Utwórz nowy projekt dla płytki `D1 (espressif8266 Module)` (lub odpowiadającej Twojemu ESP8266). Framework: `Arduino`.
Akcja 3 (Hello World - Blink): Napisz kod, który miga wbudowaną diodą
Akcja 4 (Hello Observability): Wyślij na port szeregowy “COM X” informacje o stanie diody (np. *"LED: ON"*, *"LED: OFF"*).
Akcja 5 (Push): Zrób commit i wypchnij kod na GitHub. Pamiętaj o nadaniu tagu `lab1`.

DoD (Definition of Done):

- ☐ Kod kompiluje się bez błędów w PlatformIO.
- ☐ Zbudowany kod jest wrzucony na płytkę
- ☐ Wbudowana dioda na ESP8266 miga.
- ☐ Serial Monitor wyświetla logiczny log bez "krzaków".
- ☐ Repozytorium na GitHubie zawiera poprawną strukturę PlatformIO (m.in. plik `platformio.ini`).

UKW/WI – PPC

🟡 PERFORMANCE: Naprawa (dla szybszych)

Cel: Podłączenie prawdziwego czujnika i zmiana paradygmatu – z blokującej funkcji opóźniającej na rzecz deterministycznej pętli czasowej. Nie bójcie się podejść do zrobienia PERFORMANCE 💪!

Akcja 1 (Więzy sprzętowe): Podłącz czujnik temperatury **DS18B20** do ESP8266 na płytce stykowej. Upewnij się, że obwód zawiera niezbędne elementy pasywne do poprawnej komunikacji z tym wariantem czujnika (sprawdź dokumentację/Cheat Sheet).

Akcja 2 (Deterministyczna Pętla): Zmodyfikuj kod tak, aby odczytywać temperaturę dokładnie co 2000 ms, korzystając z architektury nieblokującej. Obowiązuje całkowity zakaz używania funkcji usypiających procesor w głównej pętli.

Akcja 3 (Fail-safe & Log): Skonfiguruj logowanie do formatu analitycznego: `ts, deviceId, value, status`. Zaimplementuj mechanizm fail-safe: fizyczne odpięcie czujnika nie może "zcrashować" płytki – system ma ustawić status na `SENSOR_ERR` i działać dalej.

Akcja 4 (Parametryzacja & Health Counter): Wydziel interwał odczytu jako konfigurowalną stałą na górze pliku. Zaimplementuj *Health Counter* – wskaźnik, który rośnie po każdym błędnym odczycie, loguje tę wartość do konsoli i zeruje się po powrocie czujnika do sprawności.

DoD:

- ☐ Sprzęt zmontowany prawidłowo (bez brakujących komponentów).
- ☐ Płytka odczytuje poprawną temperaturę pomieszczenia.
- ☐ Płytka loguje dane do Serial Monitora w stałych odstępach, a otwarty kod udowadnia, że główna pętla wykonuje się bez sztucznych przerw.
- ☐ Celowe usunięcie przewodu wymusza czytelny błąd, podbija licznik awarii, a system kontynuuje pracę bez restartu.

UKW/WI - PPC

🔵 DELIGHTERS (dla ambitnych)

Cel: Samodzielne dodanie wizualnego interfejsu (Feedback) dla operatora węzła.

Akcja 1 (Sprzęt): Podłącz do płytki wyświetlacz LCD 2x16 z konwerterem I2C (podłączenie pod odpowiednie piny SDA/SCL mikrokontrolera).

Akcja 2 (Software): Znajdź i zainstaluj w PlatformIO bibliotekę do obsługi wyświetlacza po I2C.

Akcja 3 (Logika): Wyprowadź na ekran ostatnio odczytaną temperaturę w pierwszym wierszu (np. `T: 23.5 C`) oraz aktualny status pracy w drugim (np. `STATUS: OK` lub `STATUS: ERR`).

DoD:

- [ ] Ekran świeci, tekst jest czytelny, a wartości odświeżają się synchronicznie ze stanem logowanym w Serial Monitorze.

UKW/WI – PPC

⚡ Cheat Sheet

I. Architektura Wykonawcza ESP8266

Piszemy w C++ z wykorzystaniem frameworka Arduino. Oznacza to, że nasz kod opiera się na dwóch potężnych fundamentach:

- `setup()`: Uruchamiana dokładnie raz, zaraz po podłączeniu prądu. Służy do konfiguracji pinu, inicjalizacji Serial Monitora oraz startu czujników.
- `loop()`: Funkcja, która wykonuje się w nieskończoność. To tutaj musi znaleźć się nasza "deterministyczna pętla".

UKW/WI – PPC

II. Słownik Inżynierski (PlatformIO vs Klasyczne IDE)

Dlaczego odrzucamy standardowe Arduino IDE? Ponieważ uczy złych nawyków, brakuje w nim podpowiadania składni (IntelliSense) i trzyma biblioteki globalnie, psując inne projekty. Przechodzimy na standard inżynierski.

Akcja / Koncepcja	W PlatformIO (VS Code)	W inżynierskim świecie oznacza to...
Plik konfiguracyjny	`platformio.ini`	Tu definiujesz model płytki, port COM i instalujesz biblioteki. Kopia tego pliku sprawi, że projekt zadziała na każdym PC.
Kompilacja (Build)	Ikona "Ptaszka" na dolnym pasku	Sprawdzenie kodu na błędach i wygenerowanie pliku binarnego (firmware). Wymagam *0 Ostrzeżeń (Warnings)*.
Wgranie (Upload)	Ikona "Strzałki w prawo" na pasku	Fizyczne flashowanie pamięci ESP8266 nowym firmwarem przez port USB.
Monitor Portu	Ikona "Wtyczki" na pasku	Twój debugger systemowy. Pokazuje wszystko, co wyślesz poleceniem `Serial.print()`.

UKW/WI

III. Pętla bez zatrzymywania czasu

Używanie `delay(2000)` usypia procesor. Nic innego się nie wykona. Bądź inżynierem, użyj `millis()`:

```
unsigned long previousMillis = 0;
const long interval = 2000; // Interwał w milisekundach

void loop() {
  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval) {
    previousMillis = currentMillis;
    // ... Twój kod odczytu sensora
  }
}
```

UKW/WI – PPC

IV. Git Audit Trail

Pracujemy na publicznych maszynach uczelnianych. Musisz zachować dyscyplinę.

1. `git config --local user.name "Twoje Imie"` oraz `user.email`
2. `git add .` i `git commit -m "feat: stabilny odczyt z ds18b20"`
3. `git push origin main` -> Zapisujesz pracę w chmurze. Twoje jedyne zabezpieczenie.
4. Przed wylogowaniem się z komputera wpisz `git config --local --unset user.name`.

UKW/WI – PPC

V. Konfiguracja Środowiska (`platformio.ini`)

Twój plik konfiguracyjny to serce projektu. Jeśli pracujemy na płytkach **Wemos D1**, Twój plik `platformio.ini` powinien wyglądać dokładnie tak.

```
[env:d1]
platform = espressif8266
board = d1
framework = arduino

; Porty komunikacyjne (Zmień 'COM4' na port widoczny w Twoim systemie!)
upload_port = COM4
monitor_port = COM4
monitor_speed = 115200
```

Wskazówka Inżyniera: Numer portu (np. `COM3`, `COM5`) sprawdzisz w systemowym Menedżerze Urządzeń (zakładka *Porty COM i LPT*). Jeśli w Serial Monitorze widzisz "chińskie znaczki/krzaki", oznacza to, że prędkość `monitor_speed` w pliku nie zgadza się z wartością w kodzie (`Serial.begin(115200);`).

UKW/WI – PPC

VI. Hello World sprzętu: Wbudowana dioda (LED_BUILTIN)

Zanim podłączysz skomplikowane czujniki (jak DS18B20), musisz udowodnić, że potrafisz wysterować najprostszy element wykonawczy. Do tego służy niebieska dioda zamontowana bezpośrednio na Twojej płytce (w kodzie ma ona z góry przypisaną nazwę `LED_BUILTIN`).

Złota zasada Wejść/Wyjść: Mikrokontroler nie wie, co do niego podłączyłeś. Domyślnie wszystkie piny "słuchają" (są wejściami). Jeśli chcesz, żeby pin wysyłał prąd, musisz go jawnie skonfigurować wewnątrz funkcji `setup()`:

C++

```
pinMode(LED_BUILTIN, OUTPUT); // Przestawiasz pin w tryb "Wyjście"
```

Inżynierska pułapka (Logika Odwrócona / Active-Low): Jeśli uczyłeś się kiedyś klasycznego Arduino, myślisz pewnie: *HIGH = włączone, LOW = wyłączone*. **Błąd.** Na płytkach ESP8266 wbudowana dioda działa w tzw. logice odwróconej (Active-Low). Fizycznie jest ona na stałe podpięta do zasilania (3.3V), a Twój pin działa jak włącznik łączący ją z masą (GND).

Co to oznacza dla Twojego kodu w funkcji `loop()`?

- `digitalWrite(LED_BUILTIN, LOW);` ➔ Zamykasz obwód do masy. Dioda **ŚWIECI**.
- `digitalWrite(LED_BUILTIN, HIGH);` ➔ Brak różnicy potencjałów. Dioda **GAŚNIE**.

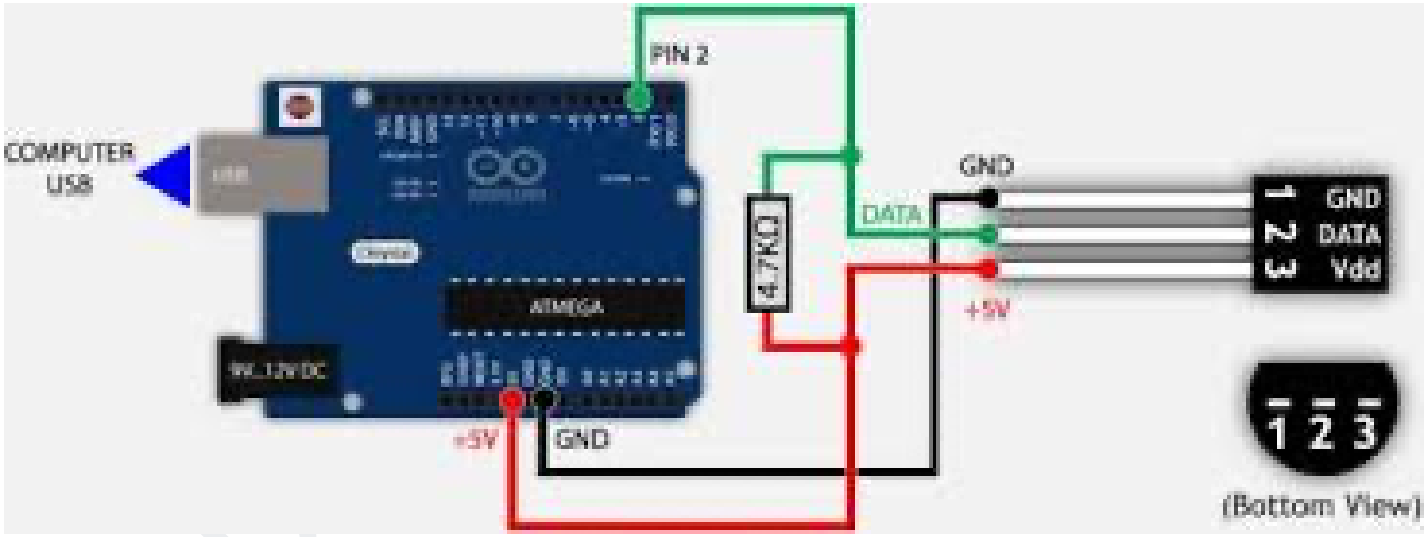
Wskazówka: Zrozumienie, dlaczego `LOW` włącza sprzęt, to pierwszy krok do zrozumienia profesjonalnej elektroniki przemysłowej, gdzie sygnały awaryjne z zasady działają w logice odwróconej, by chronić system w razie przerwania kabla.

UKW/WI - PPC

VII. Jak podłączyć DS18B20 (Złota zasada 1-Wire)

Czujnik DS18B20 komunikuje się protokołem **1-Wire**. Oznacza to, że dane i zegar idą jednym przewodem. Aby linia była stabilna, wymaga tzw. "podciągnięcia" (Pull-up) do zasilania.

- **VCC** podłącz do pinu `3V3` na ESP.
- **GND** podłącz do pinu `G` (Ground).
- **DATA** podłącz do pinu cyfrowego, np. `D4`.
- **KRYTYCZNE:** Między pinem **VCC** a pinem **DATA** musisz wpiąć na płytce stykowej rezystor **4.7kΩ**. Jeśli tego nie zrobisz, sensor zwróci wartość `-127` (co w żargonie 1-Wire oznacza "brak komunikacji z urządzeniem").



IX. Twój Warsztat Inżyniera: Instalacja PlatformIO i VS Code

Zwykłe Arduino IDE nadaje się do zabawy. My budujemy systemy, więc potrzebujemy profesjonalnego środowiska. Naszą bazą jest **Visual Studio Code (VS Code)**, z którego zrobimy potężne stanowisko dowodzenia za pomocą odpowiednich rozszerzeń (Extensions).

Krok 1: PlatformIO IDE (Twój rdzeń) To rozszerzenie zastępuje cały mechanizm Arduino – pobiera kompilatory, zarządza bibliotekami i wgrywa kod na płytkę.

1. Otwórz zakładkę **Extensions** w VS Code (ikona klocków po lewej stronie lub `Ctrl+Shift+X`).
2. Wpisz `PlatformIO IDE` (autor: PlatformIO) i kliknij **Install**.
3. ⚠️ **KRYTYCZNE:** Instalacja rdzenia (Core) dzieje się w tle. Zobaczysz powiadomienie w prawym dolnym rogu. **Nie zamykaj edytora i nie ubijaj procesu!** Może to potrwać kilka minut.
4. Gdy instalacja się zakończy, pojawi się komunikat z prośbą o restart (Reload) edytora. Zrób to. Na lewym pasku pojawi się nowa ikona (głowa mrówki/kosmity).

UKW/WI - PPO

Krok 2: "Sterydy" dla VS Code (Niezbędne dodatki) Samo PlatformIO to silnik, ale by pisać kod sprawnie i bez frustracji, doinstaluj natychmiast te 3 wtyczki:

Rozszerzenie	Autor	Dlaczego inżynier tego potrzebuje?
C/C++	Microsoft	**IntelliSense i formatowanie.** Bez tego VS Code nie zrozumie Twojego kodu. Koloruje składnię, podpowiada nazwy funkcji i dba o wcięcia.
Error Lens	Alexander	**Natychmiastowy feedback.** Brutalnie szczera wtyczka. Zamiast zmuszać Cię do szukania czerwonych falżyków, wyświetla błędy kompilacji (np. brak średnika) wielkimi literami prosto w linijce kodu. Uratuje Ci godziny debugowania.
Material Icon Theme	Philipp Kief	**Higiena plików.** Zmienia domyślne, nudne ikony plików na czytelne symbole. Od razu odróżnisz plik konfiguracyjny `.ini` od kodu źródłowego `.cpp` czy ukrytych folderów `.git`.

Złota zasada: VS Code to narzędzie, a nie czarna skrzynka. Jeśli coś podświetla się na czerwono – najedź na to myszką i przeczytaj błąd. Kompilator zawsze mówi, co jest nie tak, trzeba go tylko posłuchać.

UKW/WI - PPS

📚 Źródła i linki

Jeśli utknąłeś, nie "zgaduj". Inżynier zagląda do specyfikacji:

Poziom Zadania	Rozwiązywany Problem	Link ratunkowy (Source of Truth)
🟢 **BASIC**	Zrozumienie pętli, millis()	**[PL] "Arduino. Przepisy..."** (M. Margolis). Znajdziesz tu przepisy na deterministyczną pętlę i opóźnienia bez blokowania.
🟡 **PERFORMANCE**	Kod do DS18B20	**Random Nerd Tutorials** -> Wyszukaj: *ESP8266 DS18B20 Tutorial*. Standard rynkowy dla podłączenia tego sensora.
🟡 **PERFORMANCE**	Koncepcja Elektroniki	**Kanał Andreas Spiess** (YouTube). "Facet z akcentem". Wyjaśnia stabilność zasilania i piny ESP.
🔵 **DELIGHTER**	Obsługa ekranu po szynie I2C	**Rejestr bibliotek PlatformIO** -> Szukaj: `LiquidCrystal_I2C`. Dokumentacja powie Ci, jak zainicjalizować ekran. Pamiętaj sprawdzić na pinoucie płytki, które piny to sprzętowe **SDA** i **SCL**.

UKW / WI – PSC

UKW/WI – PPC

🎯 Na kolejny lab (LAB02 - HTTP Klient i JSON)

Cel: Skomunikowanie naszego działającego urządzenia ze światem. Zmienimy naszą płytkę w klienta sieciowego.

1. **Co musisz dokończyć w domu?** Zadania BASIC są obowiązkowe,natomiast PERFORMANCE są rekomendowane. Jeśli Wasza grupa nie zamknęła sekcji BASIC podczas warsztatu, traktujecie to jako obowiązkową pracę domową do oddania przed następnym spotkaniem. Praca w systemie jest sprawdzana na bazie commitów na GitHubie.
2. **Zapowiedź: Co będziemy budować?** Pozbędziemy się kabla USB jako jedynego źródła logów. W LAB02 ESP8266 połączy się z siecią Wi-Fi i sformatuje nasze dzisiejsze logi temperatury do nowoczesnego formatu przesyłu **JSON**.
3. **Zastanów się nad siecią:** Gdzie trzymać hasło do domowego/uczelnianego Wi-Fi, żeby nie wysłać go jawnym tekstem na GitHuba? Zacznij myśleć o pliku konfiguracyjnym, którego Git zignoruje (`.gitignore`). Będziemy o tym rozmawiać.

Widzimy się za tydzień. Znajdźcie sposób, by wasz kod był niezawodny! 💪

UKW/WI – RPC